

DS4420 - Steam Game Price Prediction: Bayesian Linear Regression

Gianna Saw and Jason Zheng

2026-04-16

Packages + Load Data

```
library(MCMCpack)

## Loading required package: coda
## Loading required package: MASS
## ##
## ## Markov Chain Monte Carlo Package (MCMCpack)
## ## Copyright (C) 2003-2026 Andrew D. Martin, Kevin M. Quinn, and Jong Hee Park
## ##
## ## Support provided by the U.S. National Science Foundation
## ## (Grants SES-0350646 and SES-0350613)
## ##

library(ggplot2)

steam_df <- read.csv("steam_clean_v2.csv")
```

Data

```
# cleaning col names --> csv files have been reading it weirdly in r
names(steam_df) <- tolower(names(steam_df))
names(steam_df) <- gsub("\\.", "_", names(steam_df))

# print out col names, just makign sure
colnames(steam_df)

## [1] "estimated_owners"      "peak_ccu"
## [3] "required_age"          "price"
## [5] "dlc_count"             "windows"
## [7] "mac"                   "linux"
## [9] "metacritic_score"      "user_score"
## [11] "positive"              "negative"
## [13] "achievements"          "recommendations"
## [15] "average_playtime_forever" "average_playtime_2weeks"
## [17] "median_playtime_forever" "median_playtime_2weeks"
## [19] "language_count"        "has_metacritic"
## [21] "has_user_score"        "genre_360_video"
## [23] "genre_accounting"      "genre_action"
## [25] "genre_adventure"       "genre_animation__modeling"
```

```

## [27] "genre_audio_production"      "genre_casual"
## [29] "genre_design__illustration"  "genre_documentary"
## [31] "genre_early_access"         "genre_education"
## [33] "genre_episodic"             "genre_free_to_play"
## [35] "genre_game_development"      "genre_gore"
## [37] "genre_indie"                 "genre_massively_multiplayer"
## [39] "genre_movie"                 "genre_nudity"
## [41] "genre_photo_editing"         "genre_rpg"
## [43] "genre_racing"                "genre_sexual_content"
## [45] "genre_short"                 "genre_simulation"
## [47] "genre_software_training"     "genre_sports"
## [49] "genre_strategy"              "genre_tutorial"
## [51] "genre_utilities"             "genre_video_production"
## [53] "genre_violent"               "genre_web_publishing"

# checking price var
summary(steam_df$price)

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.3988  0.9123  1.5239  1.5602  2.0782  6.9087

# creating price tiers, on the log scale (follows my preprocessing file)
steam_df$price_tier <- cut(
  steam_df$price,
  breaks = c(-Inf, log(0 + 1), log(5 + 1), log(15 + 1), log(30 + 1), Inf),
  labels = c("Free", "Low", "Mid", "High", "Premium"),
  right = TRUE
)

steam_df$price_tier <- as.factor(steam_df$price_tier)

# needed variables
keep_vars <- c("price_tier", "peak_ccu", "recommendations", "positive", "negative")
keep_vars <- keep_vars[keep_vars %in% names(steam_df)]

steam_df <- steam_df[complete.cases(steam_df[, keep_vars]), ]

# running other summaries, for research paper enhancement:
# data summary
table(steam_df$price_tier)

##
##      Free      Low      Mid      High Premium
##         0   66079   26228   4318    1109

prop.table(table(steam_df$price_tier))

##
##      Free      Low      Mid      High      Premium
## 0.00000000 0.67611067 0.26836106 0.04418114 0.01134713

# Train / test split (80/20)
set.seed(123)

n <- nrow(steam_df)
train_index <- sample(1:n, size = round(0.8 * n))

```

```

train_df <- steam_df[train_index, ]
test_df  <- steam_df[-train_index, ]

# Bayesian Multinomial Logistic Regression using MCMC package
# source: https://cran.r-project.org/web/packages/mcmc/mcmc.pdf
bayes_model <- MCMCmnl(
  price_tier ~ peak_ccu + recommendations + positive + negative +
    metacritic_score + achievements + average_playtime_forever + language_count,
  data = train_df,
  mcmc = 5000,
  burnin = 1000,
  thin = 5,
  verbose = 0
)

```

```

## Warning in model.response(mf, "numeric"): using type = "numeric" with a factor
## response will be ignored

```

```

## Calculating MLEs and large sample var-cov matrix.
## This may take a moment...
## Inverting Hessian to get large sample var-cov matrix.

```

```
summary(bayes_model)
```

```

##
## Iterations = 1001:5996
## Thinning interval = 5
## Number of chains = 1
## Sample size per chain = 1000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##
##              Mean          SD Naive SE Time-series SE
## (Intercept).Low      2.6719678 0.0445230 1.408e-03      1.761e-02
## (Intercept).Mid      1.8515210 0.0348887 1.103e-03      9.683e-03
## (Intercept).Premium  -1.2159733 0.0429992 1.360e-03      8.305e-03
## peak_ccu.Low         -0.4887065 0.0150741 4.767e-04      3.037e-03
## peak_ccu.Mid         -0.1611714 0.0129558 4.097e-04      2.126e-03
## peak_ccu.Premium      0.2796043 0.0289591 9.158e-04      9.543e-03
## recommendations.Low  -0.1738083 0.0108137 3.420e-04      2.726e-03
## recommendations.Mid  -0.0763650 0.0091137 2.882e-04      1.864e-03
## recommendations.Premium -0.1007254 0.0397840 1.258e-03      2.526e-02
## positive.Low         0.3177706 0.0254593 8.051e-04      8.027e-03
## positive.Mid         0.2480701 0.0196105 6.201e-04      4.323e-03
## positive.Premium     -0.3808677 0.0800854 2.533e-03      2.611e-02
## negative.Low         -0.1171456 0.0240396 7.602e-04      7.091e-03
## negative.Mid         -0.2119183 0.0219839 6.952e-04      4.595e-03
## negative.Premium      0.3510670 0.0855031 2.704e-03      2.523e-02
## metacritic_score.Low  0.0063860 0.0005792 1.831e-05      9.682e-05
## metacritic_score.Mid  0.0029956 0.0005734 1.813e-05      7.314e-05
## metacritic_score.Premium 0.0017262 0.0027135 8.581e-05      9.278e-04
## achievements.Low      0.0005373 0.0003413 1.079e-05      1.309e-04
## achievements.Mid      0.0001168 0.0003042 9.621e-06      9.486e-05
## achievements.Premium  -0.0158060 0.0019514 6.171e-05      4.522e-04

```

```

## average_playtime_forever.Low      -0.0884469 0.0105566 3.338e-04      2.224e-03
## average_playtime_forever.Mid      -0.0406498 0.0086442 2.734e-04      1.466e-03
## average_playtime_forever.Premium  0.0001191 0.0262279 8.294e-04      6.860e-03
## language_count.Low                0.0198323 0.0017006 5.378e-05      3.268e-04
## language_count.Mid                0.0058411 0.0014663 4.637e-05      1.659e-04
## language_count.Premium            0.0404367 0.0016558 5.236e-05      2.776e-04
##
## 2. Quantiles for each variable:
##
##                                2.5%      25%      50%      75%
## (Intercept).Low              2.6204449 2.6204449 2.683e+00 2.7108287
## (Intercept).Mid              1.8182901 1.8182901 1.827e+00 1.8874850
## (Intercept).Premium          -1.2515494 -1.2458510 -1.234e+00 -1.1730402
## peak_ccu.Low                 -0.5117233 -0.4948172 -4.948e-01 -0.4866170
## peak_ccu.Mid                 -0.1823780 -0.1710663 -1.591e-01 -0.1591455
## peak_ccu.Premium             0.2325313 0.2519833 2.935e-01 0.2998181
## recommendations.Low          -0.1867679 -0.1795937 -1.796e-01 -0.1696622
## recommendations.Mid          -0.0914539 -0.0803541 -8.035e-02 -0.0666138
## recommendations.Premium      -0.1448975 -0.1448975 -9.691e-02 -0.0614368
## positive.Low                 0.2704835 0.2930880 3.235e-01 0.3426810
## positive.Mid                 0.2070004 0.2323165 2.600e-01 0.2600210
## positive.Premium             -0.5058043 -0.4378697 -4.141e-01 -0.2921590
## negative.Low                 -0.1373784 -0.1373784 -1.332e-01 -0.0971162
## negative.Mid                 -0.2322310 -0.2239769 -2.240e-01 -0.1946644
## negative.Premium             0.2529631 0.2529631 3.592e-01 0.4023333
## metacritic_score.Low         0.0052021 0.0060891 6.224e-03 0.0065607
## metacritic_score.Mid         0.0015615 0.0028240 3.161e-03 0.0032172
## metacritic_score.Premium      -0.0029294 -0.0006102 2.238e-03 0.0044472
## achievements.Low             0.0002091 0.0002091 5.300e-04 0.0007041
## achievements.Mid             -0.0001662 -0.0001390 1.889e-05 0.0002354
## achievements.Premium          -0.0185909 -0.0169732 -1.697e-02 -0.0137793
## average_playtime_forever.Low  -0.1026027 -0.1016994 -8.314e-02 -0.0828510
## average_playtime_forever.Mid  -0.0539098 -0.0468926 -3.864e-02 -0.0386410
## average_playtime_forever.Premium -0.0519629 -0.0231129 1.054e-02 0.0178840
## language_count.Low           0.0167951 0.0185215 2.061e-02 0.0208791
## language_count.Mid           0.0029072 0.0052411 5.880e-03 0.0065092
## language_count.Premium       0.0376315 0.0391766 4.087e-02 0.0410363
##                                97.5%
## (Intercept).Low              2.7421307
## (Intercept).Mid              1.9122514
## (Intercept).Premium          -1.0844050
## peak_ccu.Low                 -0.4470830
## peak_ccu.Mid                 -0.1297366
## peak_ccu.Premium             0.3152069
## recommendations.Low          -0.1549930
## recommendations.Mid          -0.0594496
## recommendations.Premium      -0.0467419
## positive.Low                 0.3426810
## positive.Mid                 0.2707201
## positive.Premium             -0.2921590
## negative.Low                 -0.0667476
## negative.Mid                 -0.1592240
## negative.Premium             0.4907233
## metacritic_score.Low         0.0075651

```

```
## metacritic_score.Mid          0.0041076
## metacritic_score.Premium      0.0044475
## achievements.Low              0.0012910
## achievements.Mid              0.0008488
## achievements.Premium         -0.0116000
## average_playtime_forever.Low  -0.0666549
## average_playtime_forever.Mid  -0.0182411
## average_playtime_forever.Premium 0.0396096
## language_count.Low           0.0227513
## language_count.Mid           0.0091170
## language_count.Premium       0.0434688
```

Multinomial model, predicting on test set, confusion matrix

```
library(nnet)

# dropping the free tier model because we purposely removed this in preprocessing
train_df$price_tier <- droplevels(train_df$price_tier)
test_df$price_tier  <- droplevels(test_df$price_tier)

# multinomial model
multinom_model <- multinom(
  price_tier ~ peak_ccu + recommendations + positive + negative +
    metacritic_score + achievements + average_playtime_forever + language_count,
  data = train_df,
  trace = FALSE
)

# predicting on test set
pred_class <- predict(multinom_model, newdata = test_df)

# confusion matrix
conf_mat <- table(
  Predicted = pred_class,
  Actual = test_df$price_tier
)

conf_mat
```

```
##           Actual
## Predicted  Low   Mid  High Premium
## Low       12943 4807  771   184
## Mid        267  408  128    31
## High         0    0    2     2
## Premium      4    0    0     0
```

accuracy

```
accuracy <- mean(pred_class == test_df$price_tier)
accuracy
```

```
## [1] 0.6831227
```

Class-wise accuracy

```
prop.table(conf_mat, margin = 2)
```

```
##           Actual
## Predicted      Low      Mid      High      Premium
##   Low      0.9794914485 0.9217641419 0.8557158713 0.8479262673
##   Mid      0.0202058423 0.0782358581 0.1420643729 0.1428571429
##   High      0.0000000000 0.0000000000 0.0022197558 0.0092165899
##   Premium 0.0003027092 0.0000000000 0.0000000000 0.0000000000
```

Plots

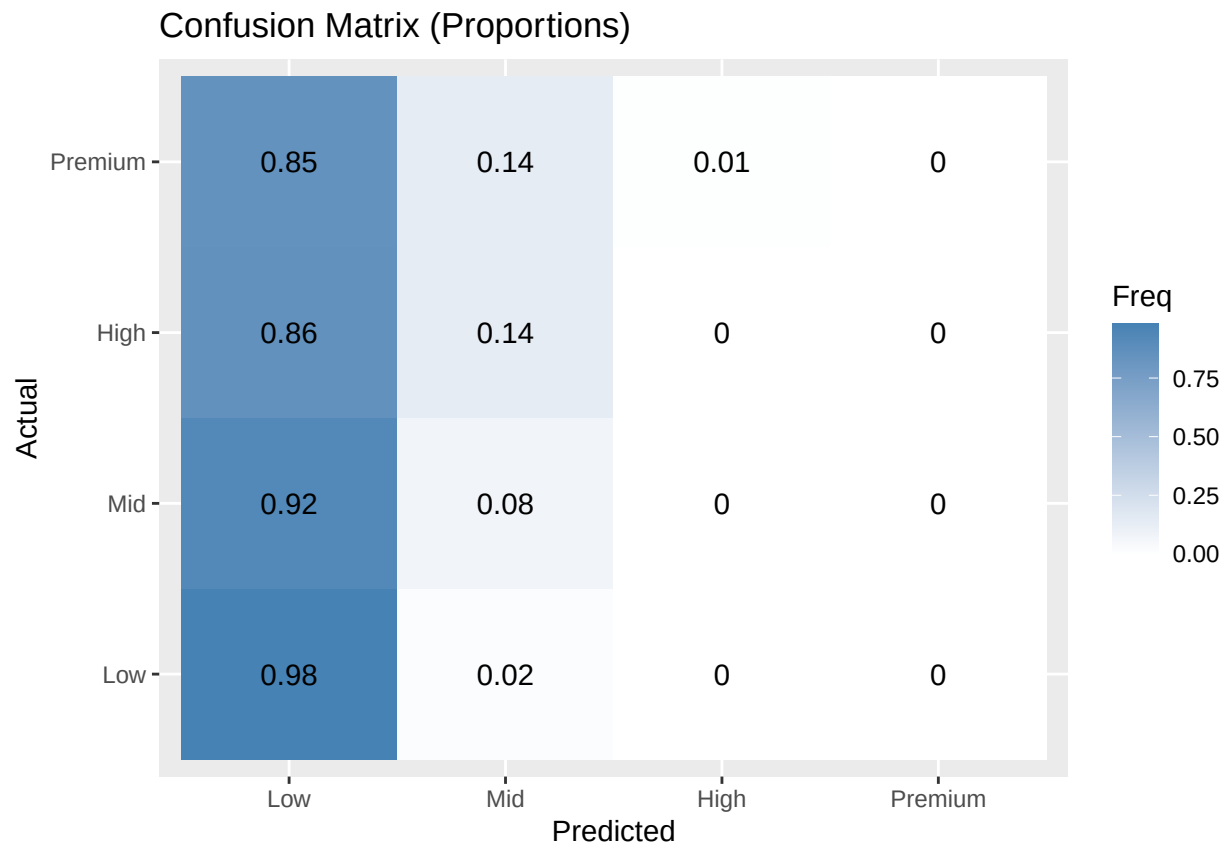
```
# Plot 1: Confusion matrix
```

```
library(reshape2)
```

```
conf_prop <- prop.table(conf_mat, margin = 2)
```

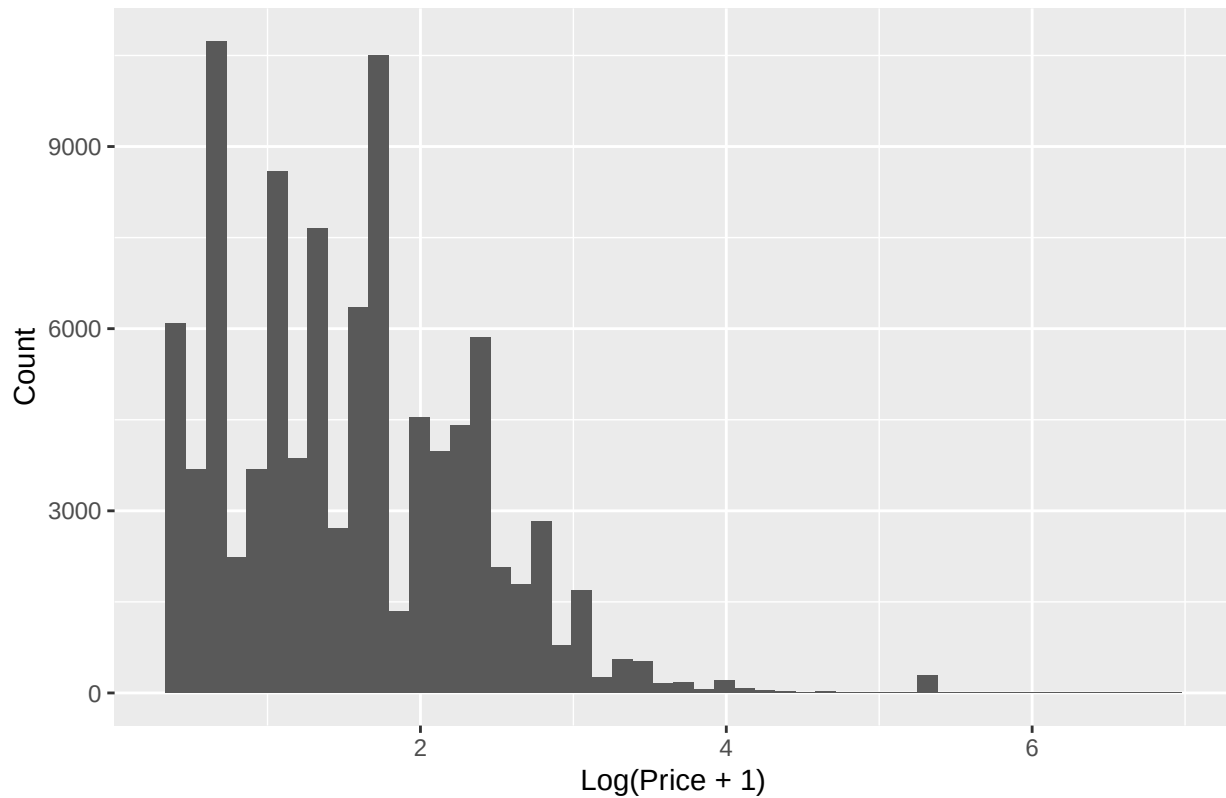
```
conf_df <- as.data.frame(conf_prop)
```

```
ggplot(conf_df, aes(x = Predicted, y = Actual, fill = Freq)) +  
  geom_tile() +  
  geom_text(aes(label = round(Freq, 2))) +  
  scale_fill_gradient(low = "white", high = "steelblue") +  
  labs(  
    title = "Confusion Matrix (Proportions)",  
    x = "Predicted",  
    y = "Actual"  
  )
```



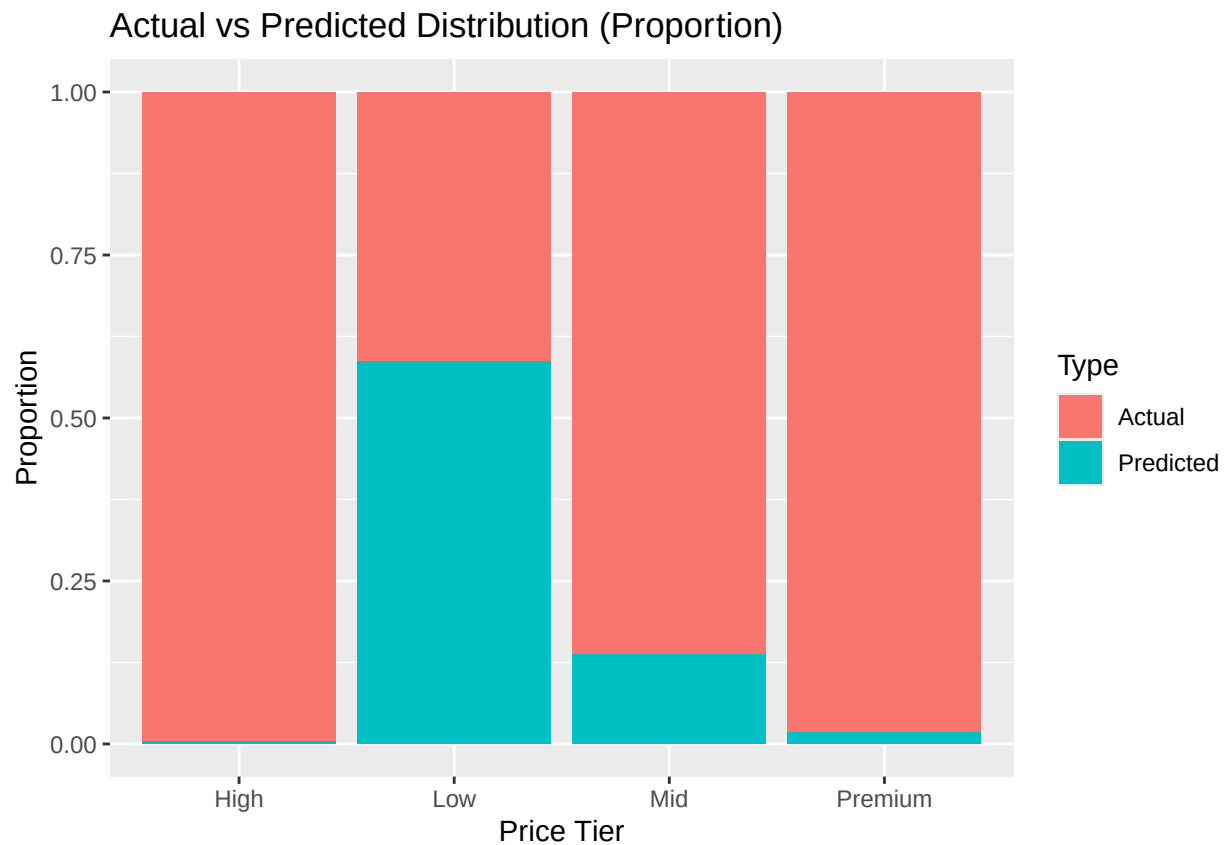
```
# Plot 2: Distribution of log-scaled game prices shows that it is skewed towards lower priced games
ggplot(steam_df, aes(x = price)) +
  geom_histogram(bins = 50) +
  labs(
    title = "Distribution of Log-Scaled Game Prices",
    x = "Log(Price + 1)",
    y = "Count"
  )
```

Distribution of Log-Scaled Game Prices



```
# Plot 3: Actual vs Predicted Distribution
plot_df <- data.frame(
  Tier = c(as.character(test_df$price_tier), as.character(pred_class)),
  Type = c(rep("Actual", nrow(test_df)), rep("Predicted", nrow(test_df)))
)

ggplot(plot_df, aes(x = Tier, fill = Type)) +
  geom_bar(position = "fill") +
  labs(
    title = "Actual vs Predicted Distribution (Proportion)",
    x = "Price Tier",
    y = "Proportion"
  )
```

```
library(nnet)
library(reshape2)

# Plot 4: Prediction confidence
pred_probs <- predict(multinom_model, newdata = test_df, type = "probs")
max_prob <- apply(pred_probs, 1, max)

ggplot(data.frame(max_prob), aes(x = max_prob)) +
  geom_histogram(bins = 30) +
  labs(
    title = "Prediction Confidence Distribution",
    x = "Max Predicted Probability",
    y = "Count"
  )
```

